

PYTHON SECURITY TOOLS

A Complete Beginner-Friendly Guide

Build your first real security tool —
step by step, concept by concept.

Author

Muhammad Usman

Certifications

OCI AI Foundation Associate · Cisco Networking: Introduction to Cybersecurity

Currently Learning

AWS Cloud Practitioner · CS50 Cybersecurity Introduction · Python Programming

Covers · Password Generation · Hashing with Salt · Modular Programming · CLI Design

■ TABLE OF CONTENTS

1. What You Are Building
2. How Python Programs Work
3. Understanding Files & Modules
4. Project Structure Explained
5. The Password Generator — Full Breakdown
6. The Password Hasher — Full Breakdown
7. The Main Program — Full Breakdown
8. How Everything Works Together
9. How to Rebuild the Project Yourself
10. Practice Exercises
11. Next Steps After This Project

■ 1. WHAT YOU ARE BUILDING

This project is a real security tool with three connected parts: a password generator, a password hasher, and a menu system that ties everything together.

- A password generator
- A password hasher
- A menu system that connects everything

This is the same modular structure found in real cybersecurity tools. By building it, you learn:

- Input handling and validation
 - Randomness and character sets
 - Cryptographic hashing
 - Modular programming
 - Returning values between modules
 - Building a CLI (Command-Line Interface) menu
-

■ 2. HOW PYTHON PROGRAMS WORK

Python runs code top to bottom, line by line. There are three core concepts your project relies on:

■ Functions — Reusable blocks of code

```
def greet():  
    print('Hello')
```

■ Modules — Python files you can import

```
from generator import generate_password
```

■ Main Program — The file that starts everything

```
if __name__ == '__main__':  
    main_loop()
```

Your project uses all three of these patterns.

■ 3. UNDERSTANDING FILES & MODULES

Your project uses three Python files, each with a single responsibility. This approach is called modular programming.

- generator.py → creates passwords
- hasher.py → hashes passwords with salt
- main.py → controls the menu and program flow

Why modular?

- Easier to read and understand
- Easier to find and fix bugs
- Easier to upgrade individual parts
- Professional software engineering practice

■ 4. PROJECT STRUCTURE EXPLAINED

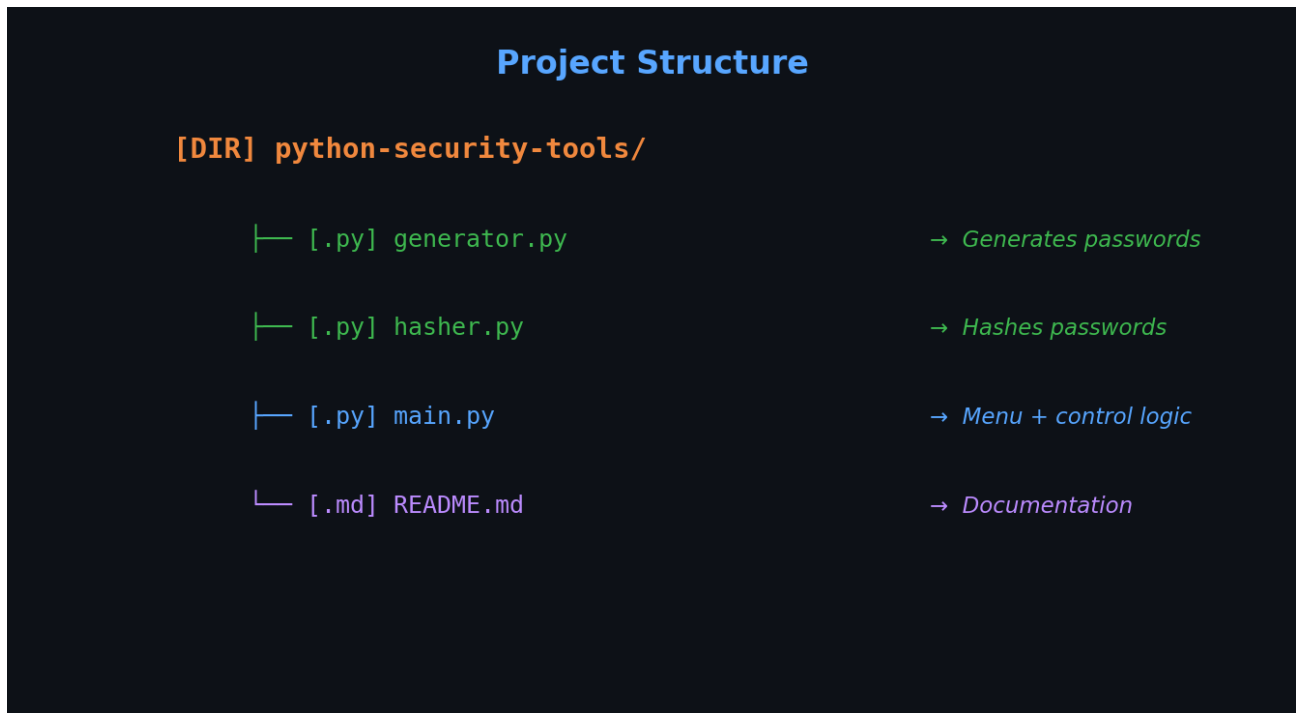


Figure 1 — Project folder structure and file responsibilities

```
python-security-tools/
├──
│   ├── generator.py    ← password creation
│   ├── hasher.py      ← hashing + salt
│   ├── main.py        ← menu and control
│   └── README.md      ← documentation
```

■ 5. THE PASSWORD GENERATOR — FULL BREAKDOWN

Here is the complete code for generator.py:

```
import random
import string

def generate_password():
    length_input = input('Enter password length: ')
    try:
        length = int(length_input)
    except ValueError:
        print('Please enter a valid number.')
```

```

    return None

if length < 8:
    print('Password must be at least 8 characters.')
    return None
elif length > 64:
    print('Password cannot be more than 64 characters.')
    return None

characters = string.ascii_letters + string.digits + string.punctuation
password = ''
for _ in range(length):
    password += random.choice(characters)
return password

```

Password Generator — Step-by-Step Flow

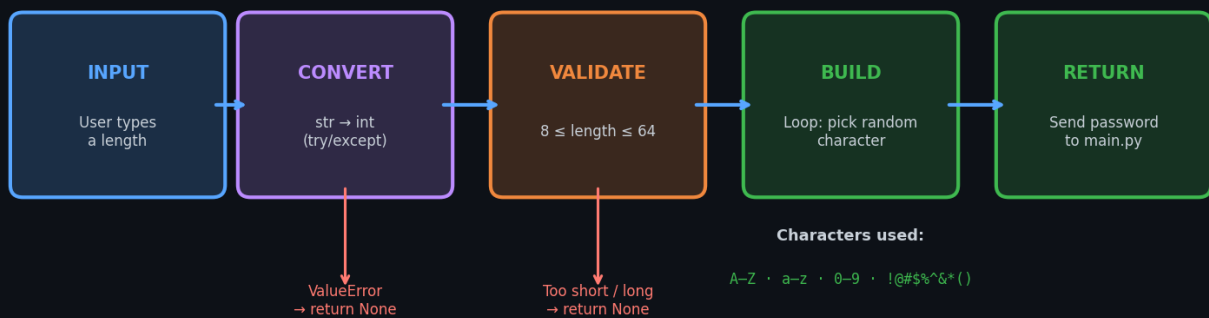


Figure 2 — Password generator step-by-step flow diagram

■ Imports

`random` gives you the ability to pick items at random. `string` gives you pre-built sets of characters: letters, digits, and symbols.

```

import random
import string

```

■ Asking for a length

`input()` pauses the program and waits for the user to type something. Everything typed is stored as a text string.

```

length_input = input('Enter password length: ')

```

■ Convert text to a number

The user's input is always text. `int()` converts it to a number. If the user types letters instead of digits, Python raises a `ValueError` — the `try/except` block catches that and returns `None` gracefully instead of crashing.

```
try:
    length = int(length_input)
except ValueError:
    return None
```

■ Validation

The generator enforces sensible limits: passwords must be at least 8 characters and no more than 64. Both limits are checked before any password is built.

■ Character set

```
characters = string.ascii_letters + string.digits + string.punctuation
```

This single line gives you every uppercase letter, every lowercase letter, every digit 0–9, and common symbols like `!@#%&*()`.

■ Generate the password

```
password = ''
for _ in range(length):
    password += random.choice(characters)
```

The loop runs exactly `length` times. On each pass, `random.choice()` picks one character at random from the character set and appends it to the growing password string.

■ Return the password

```
return password
```

Tip: returning the value (instead of just printing it) gives control back to `main.py`. This is the professional pattern — the function does one job and hands the result to whoever called it.

■ 6. THE PASSWORD HASHER — FULL BREAKDOWN

Here is the complete code for `hasher.py`:

```
import hashlib
import os
import base64

def hash_password_with_salt():
    password = input('Enter a password to hash: ')
    salt = os.urandom(16)
    salt_b64 = base64.b64encode(salt).decode()
    combined = password.encode() + salt
    hash_value = hashlib.sha256(combined).hexdigest()
    return salt_b64, hash_value
```



Figure 3 — Hashing flow: password + salt → SHA-256 → stored output

■ Imports

- `hashlib` — provides hashing algorithms including SHA-256
- `os` — provides `os.urandom()` for cryptographically secure random bytes
- `base64` — converts raw binary bytes into a readable text format

■ Generating the salt

```
salt = os.urandom(16)
```

This creates 16 bytes of cryptographically secure randomness. A salt prevents three specific attacks:

- Rainbow table attacks — pre-computed tables of known hashes become useless
- Hash collisions — identical passwords produce different hashes
- Predictable hashes — outputs cannot be guessed without the salt

■ Making the salt readable (Base64)

```
salt_b64 = base64.b64encode(salt).decode()
```

The raw salt is binary data — it cannot be stored as plain text. Base64 encodes it as a safe, readable string without altering its content.

■ Combining password and salt

```
combined = password.encode() + salt
```

Hashing only the password is weak — it produces the same hash every time. Combining the password bytes with the unique salt means every hash is different, even for the same password.

■ Hashing with SHA-256

```
hash_value = hashlib.sha256(combined).hexdigest()
```

SHA-256 is a one-way function: given any input, it always produces the same 64-character hex string — but the original input cannot be recovered from it. This is exactly how real systems store passwords.

■ Returning both values

```
return salt_b64, hash_value
```

Both the salt and the hash must be stored together. When verifying a password later, you retrieve the salt, combine it with the entered password, hash again, and compare — without ever storing the original plain-text password.

■ 7. THE MAIN PROGRAM — FULL BREAKDOWN

Here is the complete code for main.py:

```
from generator import generate_password
from hasher import hash_password_with_salt

def show_menu():
    print('\n=== PYTHON SECURITY TOOLS ===')
    print('1. Generate a secure password')
    print('2. Hash a password with salt')
    print('3. Exit')

def get_user_choice():
    choice = input('Choose an option (1-3): ')
    return choice

def main_loop():
    while True:
        show_menu()
        choice = get_user_choice()
        if choice == '1':
            password = generate_password()
            if password is not None:
                print('\nGenerated Password:')
                print(password)
        elif choice == '2':
            salt, hashed = hash_password_with_salt()
            print('\nSalt (Base64):')
            print(salt)
            print('\nSHA-256 Hash:')
            print(hashed)
        elif choice == '3':
            print('Exiting program...')
            break
        else:
            print('Invalid choice. Please select 1, 2, or 3.')

if __name__ == '__main__':
    main_loop()
```


■ Imports

main.py imports functions from the other two files. Python finds them in the same folder automatically.

```
from generator import generate_password
from hasher import hash_password_with_salt
```

■ Menu and user choice

show_menu() prints the three options each time the loop repeats. get_user_choice() reads what the user typed and returns it as a string.

■ The main loop

while True: runs forever until the user chooses option 3, which executes break to exit the loop cleanly. This is the standard pattern for CLI programs.

■ Handling each option

- Option 1 — calls generate_password() and prints the result
- Option 2 — calls hash_password_with_salt() and prints both the salt and hash
- Option 3 — prints a goodbye message and breaks the loop
- Anything else — prints an error and loops again

■ Entry point

```
if __name__ == '__main__':
    main_loop()
```

This guard ensures main_loop() only runs when you execute main.py directly. If another file were to import main.py, the loop would not start automatically.

■ 8. HOW EVERYTHING WORKS TOGETHER

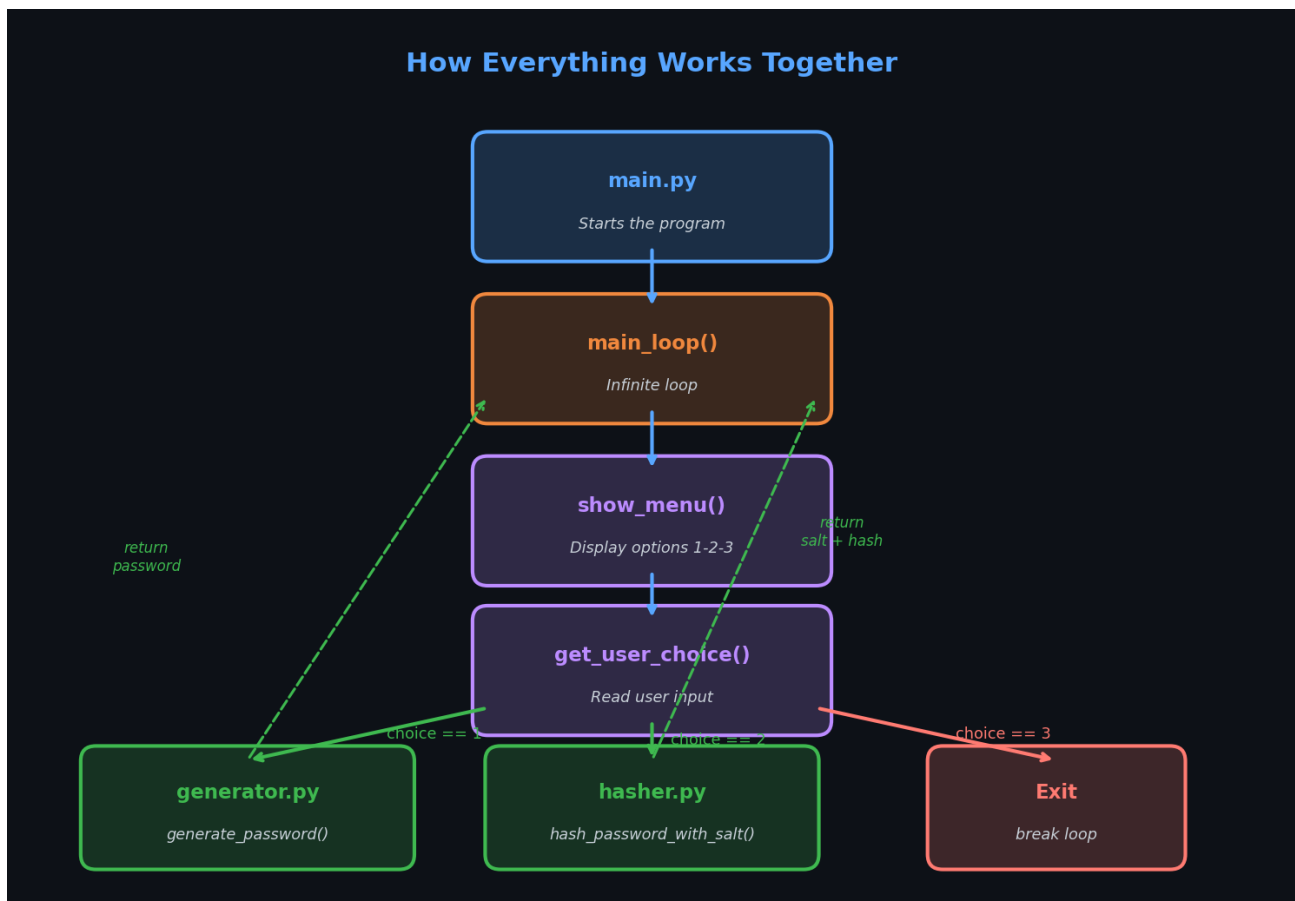


Figure 4 — Complete program flow from startup to output

When you run the program, execution starts in `main.py` and follows this path:

- `main_loop()` starts and enters an infinite while loop
- `show_menu()` prints the available options
- `get_user_choice()` reads the user's selection
- If choice is 1 → `generator.py` runs → password returned → printed
- If choice is 2 → `hasher.py` runs → salt + hash returned → printed
- If choice is 3 → loop breaks → program exits cleanly
- Any other input → error message → loop repeats

This is real modular programming. Each file has one job. `main.py` coordinates everything without doing the actual work itself.

■ 9. HOW TO REBUILD THE PROJECT YOURSELF

If you can rebuild this project from scratch without looking, you truly understand it. Follow these five steps:

✓ Step 1 — Create the folder

Make a folder called `python-security-tools`. All three files go inside it.

✓ Step 2 — Write `generator.py`

Imports → ask for length → try/except for int conversion → validate range → build password in a loop → return password

✓ Step 3 — Write hasher.py

Imports → ask for password → os.urandom(16) salt → base64 encode → combine → sha256 → return salt and hash

✓ Step 4 — Write main.py

Imports from the other two files → show_menu() → get_user_choice() → main_loop() with while True → handle 1, 2, 3 → __main__ guard

✓ Step 5 — Run it

In your terminal: python main.py

■ 10. PRACTICE EXERCISES

Beginner

- Change the minimum password length from 8 to 12
- Add option 4: 'About this tool' — print a short description
- Add colours to the menu using ANSI escape codes

Intermediate

- Save generated hashes to a text file with a timestamp
- Add a password strength meter (checks for digits, symbols, mixed case)
- Add password verification — re-hash with stored salt and compare

Advanced

- Build a GUI version using tkinter
 - Build a web version using Flask
 - Build a mobile-friendly version using a REST API
-

■ 11. NEXT STEPS AFTER THIS PROJECT

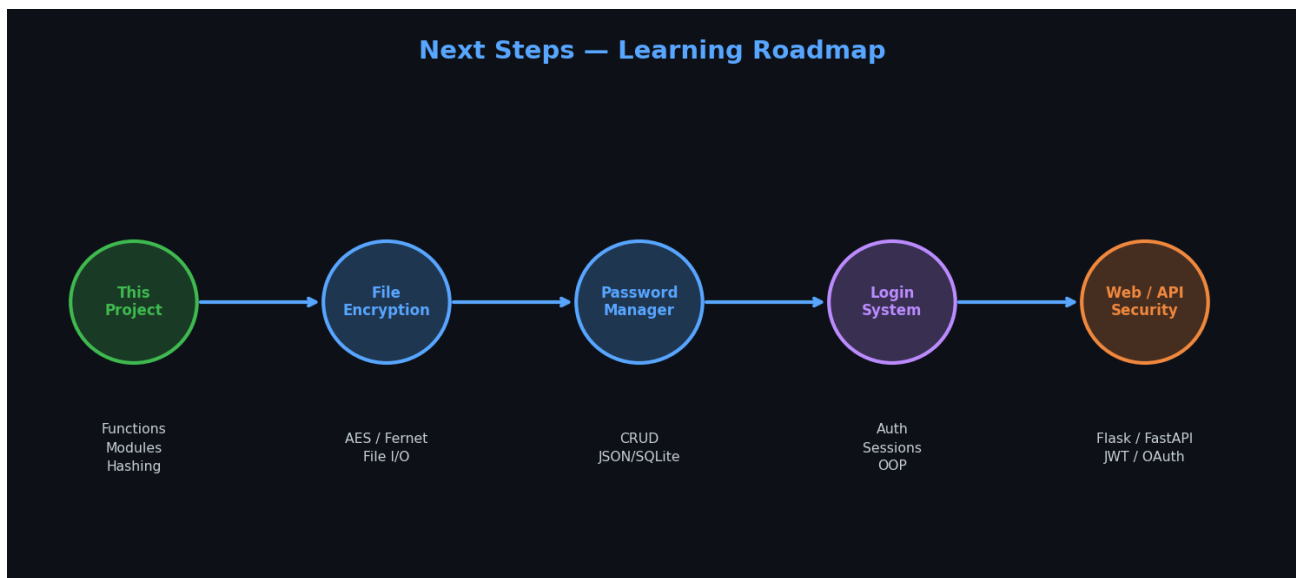


Figure 5 — Learning roadmap from this project onward

You now understand the core building blocks of security-oriented Python. These are natural next projects to build, in order of increasing complexity:

- File encryption tool — use Python's cryptography library (Fernet / AES)
- Password manager — store and retrieve hashed entries in JSON or SQLite
- Login system — full authentication with hashing and session handling
- API security basics — JWT tokens, rate limiting, input sanitisation
- Python OOP projects — rewrite this tool as a class-based architecture

Choose one project, start small, and apply the same pattern: understand first, build incrementally, test as you go.